

Requirements Engineering & Agile

Part 1: Agility

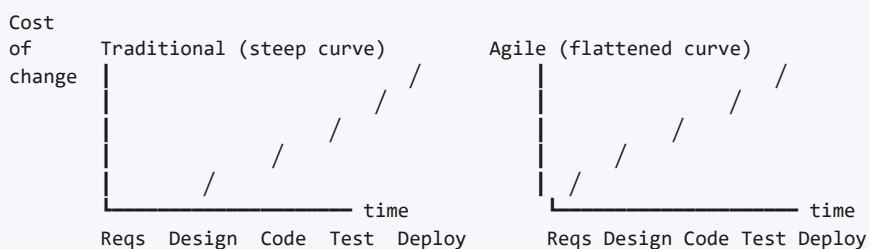
1. What Is Agility?

Agility in software engineering is the ability of a development team to respond effectively and efficiently to change throughout the software development life cycle. It promotes rapid, incremental delivery of working software, so feedback can be incorporated early and often, and it supports flexible planning — recognizing that detailed, long-term plans are hard to keep accurate in an uncertain environment.

Agility does not mean a lack of discipline. It still requires a well-defined process, continuous testing, and disciplined development practices — the difference from a traditional process is *what* the discipline is aimed at (fast, safe adaptation to change) rather than *whether* discipline exists at all. A common interview misconception is treating "agile" as "no process" — it's actually a different, still-rigorous process.

2. Agility and the Cost of Change

In traditional software development, the widely accepted rule is that **the cost of making a change increases non-linearly as the project progresses**.



- **Early changes are cheap.** A change during the requirements phase might only mean editing a document, updating a use case, or extending a list of functions — minimal effort, time, and cost.
- **Late changes are expensive.** A single change requested during validation or testing may require reworking the software architecture, redesigning and rebuilding multiple components, updating interfaces, and writing new test cases — plus there's a higher risk of introducing unintended side effects, which drives cost up even further.

Agile proponents argue that a well-designed agile process "flattens" this curve. It does this through:

- **Incremental delivery** — software is built and released in small, manageable increments, so any given change is contained within the limited scope of one increment rather than rippling through an entire, already-built system.
- **Continuous unit testing, frequent integration, and pair programming** — these catch defects early and maintain code quality continuously, reducing the ripple effect of changes and preventing the costly rework that happens when a defect is discovered long after it was introduced.

3. What Is an Agile Process?

An **agile process** is a software development process that is **adaptive, incremental, and people-centric** — designed specifically to handle the uncertainty and frequent change that occur in most real software projects.

Key Assumptions Behind Agile Processes

Agile processes are grounded in three observed realities about software projects:

1. **Uncertainty of requirements** — it's genuinely difficult to predict in advance which requirements will stay stable and which will change; customer priorities themselves shift due to market changes, feedback, or evolving understanding of their own needs.
2. **Interleaving of design and construction** — for many systems, design and coding can't be cleanly separated into sequential phases; design decisions are often only validated once construction actually begins and reveals whether the design holds up.
3. **Unpredictability of development activities** — analysis, design, coding, and testing aren't fully predictable from a planning perspective; you can't always know in advance exactly how long each will take or what each will reveal.

Because rigid, sequential processes assume the opposite of all three of these — stable requirements, cleanly separable phases, predictable activities — they tend to fail when these assumptions don't hold. This is precisely the gap agile processes are built to fill.

Need for Adaptability and Incremental Development

To manage this unpredictability, an agile process must be adaptable — achieved through incremental development, short delivery cycles, and operational prototypes or working software increments. Each increment lets the customer:

- Evaluate the software
- Provide feedback
- Influence future adaptations of *both* the product and the process itself

This iterative feedback loop is what lets adaptation keep pace with change, rather than always lagging behind it.

4. The 12 Agile Alliance Principles

The Agile Alliance defines 12 core principles that represent the agile mindset (not every agile method emphasizes all 12 equally, but together they define "the agile spirit"):

#	Principle
1	Customer satisfaction through early and continuous delivery
2	Welcoming changing requirements, even late in development
3	Frequent delivery of working software
4	Daily collaboration between business people and developers
5	Trust and support motivated individuals
6	Face-to-face communication as the most effective method
7	Working software as the primary measure of progress
8	Sustainable development, at a constant pace
9	Continuous attention to technical excellence and good design
10	Simplicity — doing only what is necessary
11	Self-organizing teams produce the best designs and architectures
12	Regular reflection and process improvement

Interview soundbite: "The 12 principles cluster into three themes: customer-facing (1, 2, 3, 7 — deliver early, welcome change, measure progress by working software), team-facing (4, 5, 6, 11, 12 — collaboration, trust, self-organization, reflection), and technical (8, 9, 10 — sustainable pace, technical excellence, simplicity). Agile isn't just 'ship fast' — a third of the principles are specifically about protecting code quality and team health."

5. Human Factors in Agile Development

Agile development strongly emphasizes **people over process** — the process is shaped around the team, not the other way around. For agility to actually succeed, the team needs these traits:

1. **Competence** — technical skills, domain knowledge, and understanding of the chosen process.
2. **Common focus** — everyone shares a single goal: delivering a working software increment on time.

3. **Collaboration** — continuous communication and cooperation among team members and stakeholders.
 4. **Decision-making ability** — the team must have real autonomy to make technical and project-related decisions, not just responsibility without authority.
 5. **Fuzzy problem-solving ability** — the team must be able to handle ambiguity, accepting that both the problem and the solution may change over time.
 6. **Mutual trust and respect** — a successful agile team is a "**jelled**" team, where trust and respect make the team stronger as a whole than the sum of its individual members.
 7. **Self-organization** — agile teams organize their own work, define their own processes, and commit to achievable goals; this improves motivation, collaboration, and morale, and makes the team genuinely accountable for its own commitments.
-

Part 2: Requirements Engineering

6. What Is Requirements Engineering?

Requirements Engineering (RE) is the systematic process of discovering, analyzing, documenting, validating, and managing software requirements.

7. The Seven Requirements Engineering Tasks

Inception → Elicitation → Elaboration → Negotiation → Specification → Validation → Management

1. Inception

The starting point of a project — begins when a business need is identified or a new market/service opportunity is discovered. The goal at this stage is to understand the problem, identify stakeholders, establish initial communication, and gain a basic idea of the desired solution.

2. Elicitation

Gathering requirements from stakeholders (customers, users, managers). This sounds simple but is genuinely difficult, because stakeholders may have unclear needs, may express requirements inconsistently, or may focus on describing *solutions* they've already imagined rather than the underlying *problems* that need solving.

A key part of elicitation is understanding **business goals**, which come in two flavors:

- **Functional** — what the system should *do*
- **Nonfunctional** — performance, security, usability, reliability

Goals help explain requirements clearly, resolve conflicts between stakeholders, and guide the later stages (elaboration, validation, negotiation). Agility matters here too, since new requirements commonly emerge during iterative development, well after the initial elicitation pass.

3. Elaboration

Creating a refined requirements model describing software functions, behavior, and information flow. This is done by developing user scenarios, identifying **analysis classes** (business entities), defining their attributes and services, and establishing relationships between classes. The objective is to understand the problem *clearly* — not to over-design. Excessive detail at this stage should be avoided; that level of detail belongs in the design phase, not requirements.

4. Negotiation

Stakeholders often request more features than available resources allow, and different stakeholders' requirements frequently conflict. Negotiation resolves this by prioritizing requirements, evaluating cost and risk, and modifying or combining requirements where possible. The goal is a **mutually acceptable solution** — no party fully "loses," and everyone reaches a reasonable level of satisfaction.

5. Specification

The formal documentation of requirements — which can take many forms: written documents, graphical models, usage scenarios, prototypes, or formal mathematical models. The right format depends on project size, system complexity, and the development environment. Large, high-assurance systems typically require a detailed **Software Requirements Specification (SRS)**; smaller systems may need nothing more than a set of usage scenarios.

6. Validation

Ensures requirements are correct, complete, consistent, unambiguous, realistic, and testable. The primary technique is a **technical review** involving developers, customers, users, and other stakeholders together. Validation is specifically what catches:

- Missing requirements
- Conflicts and inconsistencies between requirements
- Vague or unrealistic statements

Examples of problem statements caught during validation:

- *"The software should be user friendly"* — too vague to be testable; what does "friendly" mean, measurably?
- *"Intrusion probability < 0.0001"* — may be unrealistic to verify, or simply unnecessary precision for the system's actual risk profile.

Such requirements must be clarified, quantified, or replaced with more practical, testable alternatives before moving forward.

7. Requirements Management

The ongoing task of tracking requirements, their changes, their relationships to other requirements and to work products (traceability), and their status throughout the rest of the project — since requirements don't stop evolving just because specification and validation are "done" once.

8. Stakeholders and Multiple Viewpoints

A **stakeholder** is anyone who benefits directly or indirectly from the system — this includes managers, customers, end users, developers, testers, support engineers, and consultants. Each stakeholder has a different perspective, different priorities, and different risks if the project fails. At project inception, an initial stakeholder list is created and then expanded as new stakeholders are identified through the project.

Recognizing Multiple Viewpoints

Because stakeholders have different goals, requirements are gathered from multiple viewpoints:

Viewpoint	Primary Concern
Marketing	Marketable features
Business managers	Cost and schedule
End users	Usability and familiarity
Developers	Technical feasibility
Support engineers	Maintainability

These viewpoints frequently produce conflicting or inconsistent requirements, which have to be categorized and resolved to arrive at one consistent requirement set.

Working Toward Collaboration

Effective requirements engineering requires active collaboration between stakeholders and developers. The role of the **requirements engineer** is to:

- Identify requirements that are common across stakeholders
- Detect conflicts and inconsistencies between viewpoints
- Facilitate resolution through discussion or prioritization

In some cases, a designated **project champion** makes the final call when stakeholders can't agree. Techniques such as **Planning Poker** are used to prioritize requirements by having stakeholders assign points reflecting relative importance, surfacing disagreement (and its size) quickly and visibly.

Interview Questions With Answers

Q1. Explain why the cost of change increases non-linearly in traditional software development, and how agile development claims to "flatten" that curve.

Answer: In traditional development, a change requested early — during requirements — typically only requires editing a document or a use case, since nothing downstream has been built yet that depends on it. But a change requested late — during validation or testing — can require reworking the architecture, redesigning and rebuilding several components that were built around the now-outdated assumption, updating interfaces those components expose to each other, writing new test cases, and dealing with the added risk of unintended side effects from all that rework — so the *same size* of change costs dramatically more later in the project, producing the steep, non-linear curve. Agile claims to flatten this curve mainly through incremental delivery: because the system is built and released in small increments rather than as one large monolith, most changes only affect the current increment's limited scope rather than a fully-built, deeply interconnected system, and practices like continuous unit testing, frequent integration, and pair programming catch defects (and design mismatches) early, before they've had time to accumulate the kind of downstream dependencies that make late changes expensive in the first place.

Q2. What are the three key assumptions that justify why an agile (rather than rigid) process is needed, and how does each one specifically undermine a traditional sequential process?

Answer: The three assumptions are: uncertainty of requirements (it's hard to know in advance which requirements will remain stable, since customer priorities evolve with market feedback and changing understanding), interleaving of design and construction (for many systems design and coding can't be fully separated into sequential phases, since design decisions often only get properly validated once construction starts), and unpredictability of development activities (analysis, design, coding, and testing don't proceed in a fully plannable way). Each one directly undermines a rigid sequential process because that kind of process assumes the opposite in each case — that requirements are knowable and fixed up front, that design can be finished completely before any coding starts, and that the time and outcome of each phase can be accurately planned in advance; when none of those assumptions hold in practice, a process built entirely around them produces plans that are wrong almost immediately, and rigid processes have no efficient built-in way to absorb that mismatch.

Q3. Group the 12 Agile Alliance principles into rough themes and explain why "sustainable pace" and "technical excellence" are principles at all, given agile's emphasis on speed.

Answer: The 12 principles roughly cluster into customer-facing principles (early and continuous delivery, welcoming changing requirements, frequent delivery, working software as the measure of progress), team-facing principles (daily collaboration, trusting motivated individuals, face-to-face communication, self-organizing teams, regular reflection), and technical principles (sustainable pace, technical excellence and good design, simplicity). Sustainable pace and technical excellence are

included specifically as a counterbalance to the risk that "move fast and deliver continuously" gets misread as "cut corners and burn out the team" — the Agile Alliance explicitly built in principles protecting long-term code quality (technical excellence) and long-term team capacity (sustainable, constant pace) because a team that ships fast for two months by accumulating technical debt and working unsustainable hours will deliver *less* reliably over a full project's lifetime, which defeats agile's actual goal of consistently responding to change over time, not just shipping quickly once.

Q4. Why is "collaboration" listed as a human factor separately from "common focus," if both seem to be about the team working well together?

Answer: Common focus is about *what* the team is aligned on — everyone sharing the single goal of delivering a working increment on time, which is a matter of shared priorities and direction. Collaboration is about *how* the team actually works together toward that shared goal day to day — the continuous communication and cooperation between team members and stakeholders that lets information, blockers, and changing understanding flow between people in real time. A team can, in principle, share a common focus (everyone genuinely wants the same outcome) while still collaborating poorly (working in silos, not communicating changes to each other, duplicating effort) — which is exactly why agile treats them as two distinct, separately necessary human factors rather than folding one into the other.

Q5. Walk through why elicitation is described as "difficult" even though it sounds like simply asking stakeholders what they want.

Answer: Elicitation is difficult because stakeholders themselves often don't have fully clear or fully consistent needs — someone might describe a requirement one way in one conversation and a subtly different way in another, without realizing the inconsistency. It's also common for stakeholders to describe a *solution* they've already pictured in their head (e.g., "add a button that does X") rather than the actual underlying *problem* or goal that solution is meant to solve, which can lock the requirements-gathering process into one particular implementation before alternatives have even been considered. This is exactly why elicitation puts specific emphasis on separating functional and nonfunctional business goals from the stakeholder's raw statements — goals give the requirements engineer a way to check whether a stated "requirement" (which might really be a proposed solution) actually serves the stakeholder's real underlying objective, and to resolve conflicts between stakeholders by appealing to shared goals rather than incompatible proposed solutions.

Q6. A stakeholder writes the requirement "the system should be fast." Which RE task is responsible for catching this, what specifically is wrong with it, and how should it be fixed?

Answer: This is caught during the **Validation** task, whose job is specifically to ensure requirements are correct, complete, consistent, unambiguous, realistic, and testable — "the system should be fast" fails the unambiguous and testable criteria, since there's no way to write a test case that objectively determines whether the system is "fast enough" without a concrete, measurable definition. The fix is to quantify it into a specific, testable nonfunctional requirement — for example, "95% of search requests

must return results within 200ms under a load of 1,000 concurrent users" — which follows the same pattern the notes give for "the software should be user friendly": a vague qualitative requirement has to be replaced with a specific, measurable, and realistic one before it can be considered validated, typically surfaced through the technical review that Validation relies on.

Q7. Explain the difference between the Elicitation and Elaboration tasks in RE — they can sound similar since both happen early and both involve gathering information about what the system should do.

Answer: Elicitation is about *gathering* raw requirements directly from stakeholders — talking to customers, users, and managers to surface what they need, dealing with the reality that their stated needs may be unclear, inconsistent, or solution-focused rather than problem-focused. Elaboration comes after that and is about *refining* what was gathered into a structured requirements model — developing user scenarios, identifying the analysis classes (the business entities involved), and defining their attributes, services, and relationships to each other. The key distinction is raw input versus structured output: elicitation produces the (often messy) material, while elaboration organizes that material into a coherent model that's detailed enough to serve as a foundation for design, while still deliberately avoiding over-design at this stage.

Q8. Two stakeholders — Marketing and Support Engineering — have conflicting priorities for an upcoming release: Marketing wants three new customer-facing features shipped, while Support wants time spent fixing recurring issues that don't add new features. As the requirements engineer, how would you resolve this using concepts from Negotiation and stakeholder viewpoint management?

Answer: This is a textbook case for the Negotiation task, whose explicit purpose is resolving exactly this kind of situation — stakeholders requesting more than available resources allow, with genuinely conflicting priorities, where the goal is a mutually acceptable outcome rather than a full win for one side and a full loss for the other. The requirements engineer's role, per the stakeholder-viewpoint framework, is first to make each side's underlying viewpoint explicit rather than treating this as marketing-features vs. support-fixes in the abstract — marketing's viewpoint is centered on marketable features (revenue/growth impact), while support's viewpoint is centered on maintainability (ongoing cost and risk of unresolved issues) — and then to facilitate negotiation using cost/risk evaluation and possibly prioritization techniques like Planning Poker, where each stakeholder group scores items by importance, making the size and shape of the disagreement visible to everyone rather than left implicit. A realistic negotiated outcome might combine or stage the requirements — for instance, shipping the single highest-priority new feature this release while allocating a defined portion of the release to the highest-risk recurring issues — rather than an all-or-nothing choice between the two stakeholders' original requests; if no agreement is reached through this process, a designated project champion makes the final call.